

HERO1 ANDROTEXT
Editor and Compiler
IBM PC

ANDROTEXT 1.0 FOR HERO1
IBM PC Version

CONTENTS

What is ANDROTEXT?	3
Who is HERO1?	3
Rules of the Game ..	3
Getting Started	3
Branching	7
Variables	7
Assignments	8
Arithmetic Operations	8
Conditional Branches	8
Logical Operations	9
Tables	10
Data	11
For Bit Pickers Only	11
Shift Commands	11
Active Sensors	12
RANGE	13
MOTION	14
SOUND	14
LIGHT	15
TIME	15
DATE	16
TRIGGER	16
INTERRUPT	16
KEYBOARD	17
Ignoring Senses	17
Active Effectors	17
CONTINUE	20
HALT.....	21
Passive Sensors	21
Passive Effectors	22
Conditional Branches on Effectors	23
Subroutines	23
Miscellaneous Commands that Make Life Easy	25
PAUSE	25
DELAY	25
SLEEP	26
INITIALIZE	26
STOP	27
Random Numbers	27
Remarks	28

Making Your Robot Talk	28
The Incredible Adaptive Vocabulary	28
Changing the Sound of a Word	30
ANDROTEXT Editing	31
Loading Programs	31
Listing Programs	32
Hard Copies of Programs	33
Saving Programs	33
Renumbering Programs	34
Merging Programs	35
Tidying Programs	35
Pattern Seeking and Replacement in Programs	36
Opcode Editing	37
Listing Opcode	37
Hard Copies of Opcode	38
Saving Opcode	40
Loading Opcode	40
For Real Dedicated Afficionados or Pathological Masochists	41
Partial Listing	41
Pattern Seeking and Replacement in Opcode	41
Special Features	42
Compiling Long Programs	42
Compiling Humungous Programs (Link Compiling)	43
Location Maps	43
Changing Disk Drive	44
Generating a New Vocabulary File	44
Appendix 1: Display Code Characters	46
Appendix 2: ANDROTEXT 1.0 Quick Reference	47
Warranty and Copyright Information	48
Index	50

What is ANDROTEXT?

ANDROTEXT is a language to make communicating with robots easy for everyone. It is a high-level language, which means two things: (1) it is human oriented and can be understood with little technical knowledge; and (2) the same program will work on many different types of robots with minimal modification. ANDROTEXT is rapidly becoming the standard high-level language of the personal robot industry. You can be assured that the programs you write in ANDROTEXT will be useable with robots many generations from now.

Who is HERO1?

HERO1 is the first personal robot produced by the Heath Company. His senses include: (1) sound; (2) light; (3) motion; (4) ultrasonic ranging; (5) time; and (6) date. His effectors include: (1) a transport system with separate drive and steering motors; (2) a rotatable head that can direct his external sensors; and (3) a five-degree-of-freedom arm. His brain is a Motorola M6800 microprocessor, with 3746 words of useable memory. In short, Hero is magnificent.

Rules of the Game

Because ANDROTEXT is very flexible, some rules have been imposed to make it easily readable. The first and most important rule is: a single blank must separate each word of a statement. The words of a statement must contain no embedded blanks. The first word of each statement is the statement number and there should be no leading blanks. Blanks are the only separators used in ANDROTEXT: parentheses, brackets, commas, colons, and semicolons have no meaning. The second rule is: a variable cannot be used unless it has been assigned a value in a previous statement. If you violate these rules, you are likely to get an error message, but in some cases, your statements will be misinterpreted.

The best way to learn the language is to try it. So let's jump right in.

Getting Started

Install your Interface Technologies RS232 interface and ROM according to its instructions. Place your PC DOS diskette in disk drive A of your IBM PC and dead-start the system. Press the CAPS LOCK key so the PC is locked into capital letters. Type in:

DISKCOPY A: B:

PC DOS will instruct you as follows:

Insert source diskette in drive A:

Insert target diskette in drive B:

Place the diskette labeled ANDROTEXT Editor and Compiler in drive A, and a new blank diskette in drive B. Make at least two copies. Store the original and one backup in a safe place. The second copy is your working copy. You may want to copy the pertinent parts of PC DOS onto your working copy for convenience.

Connect the RS232 port of your IBM PC to the Interface Technologies RS232 port. You will need to create a null modem by crossing over the read and write lines. The Interface Technologies instruction sheet will tell you how to do this.

Place your ANDROTEXT working copy into disk drive A and enter ANDRO.

Loading the editor and compiler should take a few seconds, then your monitor screen will change to black on white with a dark blue boarder and the following text will appear.

ANDROTEXT 1.0 EDITOR AND COMPILER FOR HERO1
COPYRIGHT 1984 ROBOTRONIX, INC ALL RIGHTS RESERVED
HERO1 IS A TRADENAME OF HEATH CO.

You will also find a new set of designations for the soft keys at the bottom of the screen. These are the commands you will use most often in running the ANDROTEXT Editor and Compiler.

Now, punch in the following code on HERO's keyboard.

AA 00 85 00 07 00 11 (reset)
1D 20 17

HEROs display should go blank.

Next, type in

INITIALIZE (return)

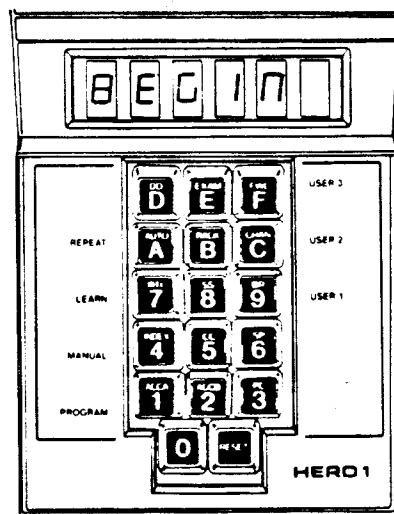
on your IBM PC. In a few seconds, HERO should say, "Ready", and return to the executive mode.

Type in the following program. (Note: Do not type the program number on the far right.)

```
10 DISPLAY *HELLO!*  
COMPILE
```

PGM[1]

While the computer is compiling, press the **9** USER 1 key on Hero's keyboard. Hero's display panel should show "BEGIN" as illustrated below.



When compilation is complete, the dark blue boarder will change to light green and the cursor will return. To transfer the compiled program to Hero, type this:

TRANSFER

The cursor will dissappear. When the transfer is complete, Hero will announce, "Ready," and the cursor will return.

Press the  USER 2 key on Hero's keyboard. He should display "HELLO!" If you've gotten this far, the rest will be easy. You're half way to understanding ANDROTEXT.

Now, type this:

NEW

This clears out the old program and readies the editor to accept a new program.

Type in this program:

```
10 DISPLAY *MARY HAD A LITTLE LAMB*  
20 DISPLAY *WHOSE FLEECE WAS WHITE AS SNOW*
```

PGM[2]

And, as before, type in:

COMPILE

Press the USER 1 key to display "BEGIN" and type TRANSFER on the computer keyboard. After Hero has signaled, "Ready", press the USER 2 key. Try it again. It's a little harder to read this, isn't it? Because Hero doesn't have a full alphanumeric display, each character can only be approximated. A complete listing of the translation of alphanumeric characters is given in Appendix 1.

The display won't show more than six characters at a time. If you need to display a long word, divide it into syllables of six or fewer characters like this:

```
10 DISPLAY *DO AN DROIDS DREAM OF ELEC TRIC SHEEP*          PGM[3]
```

If you try to display more than six characters, the computer will remind you with a syntax-error message. Try it just to see what an error message looks like.

Branching allows you to jump from one place in your program to another. Try this:

```
10 DISPLAY *TRUST YOUR ROBOT*                                PGM[4]
20 DISPLAY *HE CAN NEVER*
30 DISPLAY *FOUL UP*
40 GOTO 30
```

Compile and transfer this text. (Don't forget to press the USER 1 key.) Now, press the USER 2 key. Hero is in a loop. He won't come out of the loop until you press the RESET key.

Variables are words that represent numbers or logic values. In ANDROTEXT variables may be any length. In general, the more explicit the name the more easily the meaning of the variable can be understood. For example, "DISTANCE-TO-DOOR" could be used as a variable and is nearly self explanatory. Variables may contain letters of the alphabet and hyphens [-], but they may not contain any other special characters, numbers, or blanks. For Hero, the numerical values are the integers from 0 to 255. Try this:

```
10 NUMBER = 123                                              PGM[5]
20 DISPLAY NUMBER
```

Compile, transfer, and execute this program. The display should show "123". If it didn't work, check that you have inserted single blanks in the proper places. Statement #10 says, "Assign to the variable called NUMBER the value 123." Statement #20 says, "Display the

variable called NUMBER."

Assignments allow you to perform arithmetic or logical operations on variables or combinations of variables and constants. Constants are simply numbers whose value is fixed, like 123. Here's a little program that uses everything we have learned so far:

```
10 NUMBER = 0                                PGM[6]  
20 NUMBER = NUMBER + 1  
30 DISPLAY NUMBER  
40 GOTO 20
```

Assignments may be simple identities such as statement #10, or binary operations such as statement #20. More complex operations must be executed in several steps.

Just for fun, let's try a little variation on the last program.

```
10 N = 0                                    PGM[7]  
20 N = N + 1  
30 DISPLAY N  
40 SPEAK N  
50 GOTO 20
```

Yes, Hero is reciting the numbers. You probably want to see if he'll say, "Mary had a little lamb." He will. But let's put off that experiment until we've learned a little more.

Arithmetic Operations are accomplished by using the customary symbols:

+	addition
-	subtraction
*	multiplication
/	division

Conditional Branches allow you to branch depending on the truth of a relation. We use the following symbols:

= equal to
<> not equal to
< less than
> greater than
<= less than or equal to
>= greater than or equal to

Try this program:

```
10 N = 0
20 N = N + 5
30 DISPLAY N
40 IF N < 200 GOTO 20
50 DISPLAY *FINISH*
```

PGM[8]

This little program will make Hero go through the multiplication table.

```
100 N = 0
110 N = N + 1
120 M = 0
130 M = M + 1
140 ANSWER = N * M
150 DISPLAY N
160 DISPLAY *TIMES*
170 DISPLAY M
180 DISPLAY *EQUALS*
190 DISPLAY ANSWER
200 DISPLAY *-----*
210 IF M < 15 GOTO 130
220 IF N < 15 GOTO 110
```

PGM[9]

Logical Operations are every bit as important to robots as arithmetic operations. Here are the symbols for the usual logical operations.

AND logical "and"
OR logical inclusive "or"
XOR logical exclusive "or"
NOT logical "not"

If you are unfamiliar with logical operators refer to the truth tables given below. Given that A and B are either true (T) or false (F), here are the results of these operations.

$C = A \text{ AND } B$	{	<u>C</u>	<u>A</u>	<u>B</u>
		T	T	T
		F	T	F
		F	F	T
		F	F	F

$C = A \text{ OR } B$	{	<u>C</u>	<u>A</u>	<u>B</u>
		T	T	T
		T	T	F
		T	F	T
		F	F	F

$C = A \text{ XOR } B$	{	<u>C</u>	<u>A</u>	<u>B</u>
		F	T	T
		T	T	F
		T	F	T
		F	F	F

$C = \text{NOT } A$	{	<u>C</u>	<u>A</u>
		F	T
		T	F

Tables are words that represent lists of numbers. A table with only one number is identical to a variable. You can use tables in all the same ways as variables, except: (1) you must end all table names with a "\$"; (2) you must specify the length of a table before you use it; and (3) you must assign a number to the reserved variable ENTRY to specify which entry of the table you want to use. This program will fill table TEST\$ with numbers from 1 to 25, then Hero will speak them in reverse order. Let the program run for a while after Hero gets back to one, and see what happens.

```

10 TABLE TEST$ 25
20 ENTRY = 0
30 ENTRY = ENTRY + 1
40 TEST$ = ENTRY
50 IF ENTRY < 25 GOTO 30
60 ENTRY = ENTRY - 1
70 SPEAK TEST$
80 GOTO 60

```

PGM[10]

After Hero reached the bottom of the table he started reading numbers from strange places in his memory. There is nothing to keep you from reading garbage when you run off the end of a table. Similarly, you can clobber good data if you write off the end.

Statement #10 tells the compiler that you will use a table named TEST\$ and its length will be 25 numbers. You must specify all table lengths before you use any tables or variables, so it's a good idea

to do it at the beginning of your program. You can use as many tables as you like, but the total of all the lengths must not exceed 256.

Data may be entered into a table at compilation time. Data will remain in the table until another value is assigned. Here's an example.

```
10 TABLE PRIME$ 30                                PGM[11]
20 DATA 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59
30 DATA 61 67 71 73 79 83 89 97 101 103 107 109 113
40 ENTRY = 0
50 ENTRY = ENTRY + 1
60 DISPLAY PRIME$
70 IF ENTRY < 30 GOTO 50
```

As the summed length of all tables may not exceed 256, neither may the summed length of data statements. Data statements may appear anywhere in a program, but it is best to have them immediately follow the table to which they apply.

For Bit Pickers Only

If you're familiar with computers, you know that an eight-bit variable can contain eight different trues and falses. Some people call this a logic vector, although I can't imagine why. We might say $A = \text{TFFTFTFF}$, or $A = 10010100$, which in our usual decimal notation is $A = 148$. By simply saying $A = \text{TRUE}$, we get $A = \text{TTTTTTT} = 1111111 = 255$; awfully redundant, and a waste of space. Similarly, $A = \text{FALSE}$ gives us $A = \text{FFFFFFF} = 00000000 = 0$.

If you want to take the time to set each bit in a logic vector, ANDROTEXT provides a few ways to make it easy.

Shift Commands : Variables can be shifted to the right or left. Try this:

```
1 A = 1                                PGM[12]
2 N = 0
3 B = A LEFT N
4 DISPLAY B
5 N = N + 1
```

6 GOTO 3

The results should have been: 001 (no shift); 002 (shift one bit left); 004 (shift two bits left); 008 (shift three bits left); etc. Each time A is shifted, bits fall off the left end, and the right end is filled in with zeros. Once A gets to zero, it stays there forever. Similarly, there is a shift to the right; try this:

```
1 A = 128
2 N = 0
3 B = A RIGHT N
4 DISPLAY B
5 N = N + 1
6 GOTO 3
```

PGM[13]

This should have done just the opposite. We use the shift commands to park bits in selected positions in a variable, and to pick them out again. For example, to set the fourth bit of A, we might write:

```
50 B = 1
60 B = B LEFT 4
70 A = A OR B
```

And to test the fourth bit of A, we could write:

```
40 B = A RIGHT 4
50 B = B AND 1
60 IF B = 1 GOTO .....
```

This may not seem very useful now, but, when you use logic on tables of numbers, the utility of the shift command will become clear.

Active Sensors

The mechanisms by which the robot receives information about the outside world and his relationship to the world are sensors. The active sensors are those that the robot must be told to activate, and

the inclusion of which entails use of additional robot memory. Most active sensors require more power, and you won't want to use them if you don't have to. When activated, each of these stores quantities in one or more reserved variables that can be used like any other variable in your program. The table below lists Hero's active senses and their corresponding reserved variables.

<u>Sense</u>	<u>Reserved Variables</u>
RANGE	RANGE, HITS
MOTION	MOTION
SOUND	SOUND
LIGHT	LIGHT
TIME	HOUR, MINUTE, SECOND
DATE	YEAR, MONTH, DAY
TRIGGER	TRIGGER
INTERRUPT	INTERRUPT
KEYBOARD	KEYBOARD

Each of these senses returns information in its reserved variables, but each is slightly different, so here is a brief discription of each.

RANGE uses Hero's sonar to detect and determine the distance to a physical object or obstruction. The reserved variable RANGE will contain the distance corresponding to the most recent echo detected. RANGE is determined by measuring the time interval between firing the sonar pulse and receiving its echo. The reserved variable HITS returns the number of echos received. HITS is set to zero when the robot is instructed to SENSE RANGE and is increased by one each time an echo is received. Here's a program that will display the value of range and will display "CLOSE!" when an object is closer than 1 foot.

```
10 SENSE RANGE                                PGM[14]
20 IF RANGE < 30 GOTO 50
30 DISPLAY RANGE
40 GOTO 20
50 DISPLAY *CLOSE!*
60 GOTO 20
```

RANGE is given in centimeters, although the measurement is dependent on altitude, barometric pressure, and temperature, and may vary

slightly from day to day. The distance is measured from a point about 10 centimeters from the axis of HERO's head and will be slightly exaggerated at close range because of the distance between the transmit and receive tubes.

One inch is 2.54 centimeters. The following program will give a corrected distance in inches from the axis of HERO's head. It illustrates the use of arithmetic operations to condition data from Hero's sensors. See if you can figure it out.

```
10 SENSE RANGE                                PGM[15]
20 D = RANGE + 10
30 A = D / 2
40 B = D / 11
50 DISTANCE = A - B
60 DISPLAY DISTANCE
70 GOTO 20
```

MOTION uses Hero's Doppler shift detector to signal the presence of anything moving in his immediate vicinity. The reserved variable MOTION returns the number of times movement has been detected. It is set to zero when Hero is instructed to SENSE MOTION and is increased by one each time motion is sensed. Like any other variable, you can set it to any value you like, but as long the motion detector is on, it will continue to be increased by one each time motion is detected. In this way, it is just like HITS.

Here's a program that will wait until Hero has sensed motion 20 times, which will surely happen only when someone is in his presence. He will then display "HELLO!", and reset himself.

```
10 SENSE MOTION                                PGM[16]
20 DISPLAY *-----*
30 IF MOTION < 20 GOTO 30
40 DISPLAY *HELLO!*
50 MOTION = 0
60 GOTO 20
```

SOUND senses the ambient sound level and returns it as numerical value in the reserved variable SOUND. Like MOTION, you can set SOUND to any value you like, but it will be instantly changed back as long

as Hero is sensing sound. Here's a program to illustrate a simple use of this sense.

```
10 SENSE SOUND
20 IF SOUND > 100 GOTO 50
30 DISPLAY SOUND
40 GOTO 20
50 DISPLAY *OUCH!!*
60 GOTO 20
```

PGM[17]

When Hero is first instructed to SENSE SOUND, the compiler automatically inserts a delay of one second, so the detector will settle and not return a false signal.

LIGHT returns the ambient light level in the reserved variable LIGHT. Hero's architecture does not permit him to sense light and sound at the same time. This program alternately displays the light level and the sound level.

```
10 SENSE LIGHT
20 DISPLAY *LIGHT*
30 DISPLAY LIGHT
40 SENSE SOUND
50 DISPLAY *SOUND*
60 DISPLAY SOUND
70 GOTO 10
```

PGM[18]

TIME reads the clock and sets the reserved variables HOUR, MINUTE, and SECOND. These values are set at the instant HERO is instructed to SENSE TIME. The variables will not change again unless your program explicitly changes them, or you instruct Hero to SENSE TIME again. Frequently, you will want to use Hero's sense of time to cause Hero to undertake tasks at calculated or predetermined intervals, which will require Hero to perform arithmetic operations on the reserved variables. Here's a program will make Hero display "FIFTH" every fifth second. See if you can figure it out.

```
10 SENSE TIME
20 FIFTH = SECOND / 5
30 FIFTH = 5 * FIFTH
```

PGM[19]

```
40 FIFTH = SECOND - FIFTH
50 IF FIFTH = 0 GOTO 10
60 DISPLAY *FIFTH!*
70 DISPLAY *-----*
80 GOTO 10
```

This program illustrates three things: (1) how the sense of time can be used in arithmetic operations; (2) how the peculiar properties of integer arithmetic can be used to your advantage; and (3) how repetitive use of the same variable in different contexts can make a program difficult to read.

DATE reads the calendar and gives the date in the same way TIME gives the time. See if you can figure out this program.

```
10 SENSE DATE
20 IF DAY <> 1 GOTO 10
30 IF MONTH <> 1 GOTO 10
40 DISPLAY *HAPPY NEW YEAR*
50 GOTO 10
```

PGM[20]

TRIGGER counts the number of times the trigger switch on Hero's teaching pendant makes a connection. It is similar to the motion sense, because the reserved variable TRIGGER can be reset at any time. The TRIGGER sense is not "debounced", which means that it may count several times during the pull of the trigger because the switch may make several connections on a short time scale. It is left the way because you may want to connect other apparatus to Hero's teaching pendant port for which debouncing is not useful.

Try this program:

```
10 SENSE TRIGGER
20 DISPLAY *QUIET*
30 IF TRIGGER = 0 GOTO 20
40 DISPLAY *SIGNAL*
50 GOTO 10
```

PGM[21]

INTERRUPT works just like the trigger sense, but uses the interrupt connection on the experimental board. If you want to experiment with

the interrupt sense, try using the above program with the word INTERRUPT in place of the word TRIGGER. Be sure to remove the interface board before sending a signal to the interrupt.

KEYBOARD senses a number punched in on the robot's keyboard and puts it into the reserved variable KEYBOARD. This little program will have HERO recite each number you punch in on his keyboard.

```
10 SENSE KEYBOARD
20 SPEAK KEYBOARD
30 GOTO 10
```

PGM[22]

Notice that numbers greater than 255 cannot be interpreted, nor can alphabetic characters.

Ignoring Senses: As you can instruct your robot to sense certain aspects of his environment, you can also instruct him to ignore them. You may do this as a convenient way to turn the senses on and off, or you may want to be parsimonious in your use of the senses to conserve your robot's energy supply. The allowable instructions for Hero are:

```
IGNORE RANGE
IGNORE MOTION
IGNORE SOUND
IGNORE LIGHT
IGNORE TRIGGER
IGNORE INTERRUPT
IGNORE KEYBOARD
```

There need be no corresponding instruction for TIME and DATE, because these are one-time operations.

Active Effectors

Effectors are those mechanisms by which a robot can change the outside world or his relationship to it. Active effectors are those that involve mechanical action, as opposed to the simple transmission of information. For active effectors, ANDROTEXT offers a great deal of flexibility. A low level of artificial intelligence is employed to allow your robot to understand almost any unambiguous statement

that would be intelligible to a human reader.

All Hero's active effectors are all electric motors. Frequently, you will want to be quantitative in your instructions to an effector. Each motor has a numerical range and starting position.

<u>Motor</u>	<u>Range</u>	<u>Initial</u>
Arm Extend/Retract	152	0
Arm Pivot	134	0
Drive	1023	0
Gripper	117	0
Head	191	98
Steering	152	73
Wrist Pivot	181	0
Wrist Rotate	147	77

All Hero's motors must be initialized before they can be used. You can accomplish this by pressing 31 on Hero's keyboard immediately after turning him on or anytime a program finishes with the motors out of their initial positions.

Let's look at some typical effector commands. This command will make Hero drive forward to the end of his range.

10 DRIVE FORWARD

This makes Hero drive forward 25 steps from where he is:

10 DRIVE FORWARD 25

This makes Hero go to absolute position 525:

10 DRIVE FORWARD TO ABSOLUTE POSITION 525

Now, in each of these it would have been possible to tell Hero something else and be equally effective at communicating. For example, GO FORWARD, MOVE FORWARD, ROLL FORWARD, and WALK FORWARD

would all be interpreted the same way, because the word FORWARD removes all ambiguity. Hero will however balk at such phrases as "press on" or "jive out."

Here are some examples of phrases that will be properly interpreted:

DRIVE BACKWARD
FORGE AHEAD 20 STEPS
RETRACT ARM TO 40
EXTEND ARM 14 STEPS
EXTEND ARM QUICKLY TO 45
PIVOT ARM UP AT MEDIUM SPEED
PIVOT ARM TO 75
PIVOT ARM DOWN
PIVOT ARM UP
ROTATE WRIST LEFT
ROTATE WRIST QUICKLY RIGHT TO 44
QUICKLY ROTATE WRIST
DRIVE FORWARD FAST
MARCH FORWARD AT QUICK TIME
BACK UP FAST
CLOSE GRIPPER
OPEN GRIPPER
SLOWLY OPEN GRIPPER 15 UNITS
TURN YOUR HEAD LEFT 31 STEPS
SLOWLY TURN HEAD LEFT
HEAD TO 62
EYES RIGHT
PULL BACK 3 STEPS
STEER LEFT
STEERING TO POSITION 26 AT SLOW SPEED
TURN STEERING LEFT 14 STEPS
QUICKLY STEER RIGHT

The best way to learn how to communicate with your robot is to try a few commands and see what happens. Before long you will have developed a rapport and a lasting friendship.

To demonstrate the use of effectors in conjunction with sensors, here is a primitive little program that will allow Hero to blunder his way around the room. He will probably get in trouble, so be prepared to bail him out.

```
10 SENSE RANGE
20 SLOWLY MOVE FORWARD 10 STEPS
30 IF RANGE > 50 GOTO 20
40 STEER LEFT 70 UNITS
50 RANGE = 255
60 SLOWLY PULL BACK 10 STEPS
70 IF RANGE < 75 GOTO 50
80 STEER RIGHT 70 UNITS
90 GOTO 20
```

PGM[23]

Right now you are probably wondering whether a variable can be used in effector commands in lieu of a numerical value. Certainly, here's an example:

```
1 X = 0
2 DISPLAY X
3 EXTEND YOUR ARM X UNITS
4 NOW RETRACT IT X
5 X = X + 1
6 IF X < 153 GOTO 2
```

PGM[24]

You can do the same sort of thing with tables. Here's a purely illustrative example:

```
100 TABLE X$ 5
110 DATA 10 20 30 40 50
120 TABLE Y$ 5
130 DATA 75 25 95 45 55
140 ENTRY = 0
150 ENTRY = ENTRY + 1
160 TURN HEAD TO POSITION X$
170 EXTEND ARM TO POSITION Y$
180 IF ENTRY < 5 GOTO 150
190 GOTO 140
```

PGM[25]

See if you can make Hero touch the corners of a square just by changing the data statement in this program.

CONTINUE: Having given the robot an effector command, we can either

wait for that command to be completed, or continue to execute the program while that command is being carried out. To continue execution, a CONTINUE statement must immediately follow the effector command.

HALT: Any effector that is allowed to continue can be stopped with the HALT command. For the explicit effectors these are:

HALT ARM
HALT DRIVE
HALT GRIPPER
HALT STEERING
HALT WRIST

Halting the arm, of course, also halts movements of the wrist and gripper. We say the arm, gripper, and wrist are multiplexed.

This simple program, a variation of the previous one, illustrates the use of effector commands, HALT, and CONTINUE, in conjunction with sensors. It is the same as the previous program except, when Hero gets into trouble, you can stop him by shouting, "Halt."

```
100 SENSE RANGE                                PGM[26]
110 SENSE SOUND
120 RANGE = 255
130 DRIVE FORWARD 1 STEP
140 CONTINUE
150 IF SOUND > 100 GOTO 210
160 IF RANGE > 100 GOTO 120
170 STEER LEFT 20
180 SLOWLY BACK 10 STEPS
190 STEER RIGHT 20
200 GOTO 120
210 HALT DRIVE
220 DISPLAY *HALT*
```

Passive Sensors

Passive sensors communicate information about the inner state of the robot. Most important of these are the kinesthetic senses, which reveal the state of the active effectors. These sensors are always working, just like your own kinesthetic senses and metabolic

regulators. The reserved variables for these senses are:

ARM-EXTEND
ARM-PIVOT
GRIPPER-OPEN
HEAD-ANGLE
STEERING-ANGLE
WRIST-PIVOT
WRIST-ROTATE

The passive sensors provide ways of feeding information into your program that would be difficult to acquire by other means.

Here's a little program that may seem silly, but illustrates an important principle. It will make Hero extend his arm in proportion to the light level

```
10 SENSE LIGHT                                PGM[27]
20 IF LIGHT > ARM-EXTEND GOTO 100
30 IF LIGHT < ARM-EXTEND GOTO 200
40 GOTO 20
100 EXTEND ARM 1 UNIT
110 GOTO 30
200 RETRACT ARM 1 UNIT
210 GOTO 20
```

This program shows how passive-sensor feedback loops can be used in a robot's programming. It is very similar to how your own kinesthetic sense guide the movement of your muscles. You will find this principle very useful.

Passive Effectors

Passive effectors only transmit information to the outside world. For Hero the only passive effectors are DISPLAY and SPEAK. We have already learned about DISPLAY and will discuss SPEAK later. Like the other effectors these can be stopped with the HALT command. The proper commands are:

HALT DISPLAY
HALT SPEAK

Conditional Branches on Effectors

Because your robot's brain can function while effector commands are being executed, you will want to use information on the state of the effectors to make decisions within your programs. Conditional branches are provided, which depend on whether an effector is IDLE or BUSY. These are:

IF ARM IS BUSY...	IF ARM IS IDLE...
IF DRIVE IS BUSY...	IF DRIVE IS IDLE...
IF GRIPPER IS BUSY...	IF GRIPPER IS IDLE...
IF HEAD IS BUSY...	IF HEAD IS IDLE...
IF STEERING IS BUSY...	IF STEERING IS IDLE...
IF WRIST IS BUSY...	IF WRIST IS IDLE...

For Hero, HEAD and ARM are multiplexed, as are ARM, WRIST, and GRIPPER. Here's a simple example to illustrate the use of effector branches. Hero will display his arm position as long as his arm is moving.

```
100 PIVOT ARM UP
110 CONTINUE
120 IF ARM IS IDLE GOTO 150
130 DISPLAY ARM-PIVOT
140 GOTO 120
150 DISPLAY *UP*
160 PIVOT ARM DOWN
170 CONTINUE
180 IF ARM IS IDLE GOTO 210
190 DISPLAY ARM-PIVOT
200 GOTO 180
210 DISPLAY *DOWN*
220 GOTO 100
```

PGM[28]

Subroutines

If a portion of your program is used frequently and is branched to from several different places, you may want to write it as a subroutine. You branch to a subroutine by using the word GOSUB followed by the statement number of the beginning of your subroutine. This part is exactly like GOTO. The difference is that at the end of your subroutine you place the word RETURN. This tells your robot to return to the statement immediately following the GOSUB.

Here's a very simple example, which will make your robot alternately start or stop each time you yell at him.

```
10 GOSUB 100                                PGM[29]
20 SLOWLY DRIVE FORWARD
30 DISPLAY *GO!*
40 GOSUB 100
50 HALT DRIVE
60 DISPLAY *HALT!*
70 GOTO 10
100 SENSE SOUND
110 IF SOUND < 30 GOTO 110
120 RETURN
```

The following program will make HERO scan the horizon and display the range and bearing of the closest object. It illustrates the use of multiple subroutines.

```
100 SENSE RANGE                                PGM[30]
110 MINIMUM = 255
120 SCAN RIGHT 2 UNITS
130 GOSUB 210
140 IF HEAD-ANGLE < 180 GOTO 120
150 GOSUB 260
160 SCAN LEFT 2 UNITS
170 GOSUB 210
180 IF HEAD-ANGLE > 10 GOTO 160
190 GOSUB 260
200 GOTO 120
210 IF RANGE < 10 GOTO 250
220 IF RANGE > MINIMUM GOTO 250
230 MINIMUM = RANGE
240 BEARING = HEAD-ANGLE
250 RETURN
260 DISPLAY *RANGE*
270 DISPLAY MINIMUM
280 DISPLAY *BEARNG*
290 DISPLAY BEARING
300 MINIMUM = 255
310 RETURN
```

Bearing is given in units of the reserved variable HEAD-ANGLE. See if you can modify this program to give relative bearing in degrees.

Miscellaneous Commands that Make Life Easy

PAUSE, followed by a number, tells Hero how many seconds to wait before executing the next statement. The number cannot be greater than 4000, which is nearly three days. Here's a program that will make Hero wait 10 seconds before displaying "HELLO".

```
10 PAUSE 10
20 DISPLAY *HELLO!*
```

PGM[31]

PAUSE is most useful when relatively long waits are necessary between stages of a program. (Note: Hero's architecture will not allow PAUSE to be used while TIME or DATE are being sensed.)

DELAY is just like PAUSE, but the units are milliseconds. Here's a program that does the same thing as the one above.

```
10 DELAY 10000
20 DISPLAY *HELLO!*
```

PGM[32]

DELAY is 1000 times faster than PAUSE, and cannot exceed 50000 milliseconds, which is 50 seconds.

DELAY is particularly useful for coordinating your robot's sensors with the real-time environment. The following example will illustrate this.

In the science fiction movie The Day The Earth Stood Still, our extraterrestrial visitor, Klatu, uses a borrowed flashlight to signal his giant robot, Gort. Gort has been standing motionless for many days, but ponderously grinds into action when he sees Klatu's secret signal. The below program remains in a tight loop until it detects an increase in light level of more than one unit. It then uses a loop on the DELAY command to see if there are five more similar flashes within five seconds. It doesn't matter what the ambient level is, so the program should function as well in a bright room as in a dark room. Like Klatu, you can activate your robot by shining

your flashlight on a wall in front of him, and remember, "Klatu barrada nichto!"

```
100 SENSE LIGHT
110 HALT DISPLAY
120 FLASH = 0
130 TEST = LIGHT + 1
140 IF LIGHT < TEST GOTO 140
150 TIME = 0
160 TEST = LIGHT + 1
170 DELAY 100
180 TIME = TIME + 1
190 IF TIME = 50 GOTO 120
200 IF LIGHT < TEST GOTO 160
210 FLASH = FLASH + 1
220 IF FLASH < 5 GOTO 160
230 DISPLAY *HELLO KLATU*
240 TURN HEAD TO 180
250 TURN HEAD TO 98
260 GOTO 110
```

PGM[33]

It's easy to elaborate on this, and set up any kind of signal sequence you want. You can even construct secret codes using tables. Try writing a program that uses sound instead of light.

SLEEP puts the robot in an energy saving sleep. For Hero, it will only work with the Sleep/Normal switch in the Sleep position. The units are minutes, so this program will make Hero sleep for a half hour.

```
10 SLEEP 30
20 DISPLAY *HELLO!*
```

PGM[34]

INITIALIZE does the same thing as 31 on Hero's keyboard. It sets all effectors to their initial positions. Try this:

```
10 INITIALIZE
20 DISPLAY *DONE*
```

PGM[35]

STOP halts execution and returns the robot to the executive mode. Try this:

```
10 DISPLAY *HELLO!*
20 STOP
30 GOTO 10
```

PGM[36]

It didn't loop, did it? There's an implied STOP at the end of every program, so the STOP command is most useful for halting execution in the middle of a program.

Random Numbers: ROLL initializes the random number generator, which puts random numbers in the reserved variable RANDOM. A different random number will appear every time RANDOM is used. Here's a program that will generate and speak random numbers:

```
10 ROLL
20 SPEAK RANDOM
30 GOTO 20
```

PGM[37]

Random numbers are most useful for making arbitrary decisions on the basis of probabilities. The following program uses the DELAY command and a conditional branch on the drive motor to determine whether Hero has run into something. If he has, he will back up and turn right or left with equal probability. David Heiserman calls this an alpha level program: it makes a decision on the basis of no prior experience. If the program kept a table of probabilities from prior experience, it would be beta level, and if it could generalize, it would be gamma level.

```
100 ROLL
110 SLOWLY GO FORWARD 10 STEPS
120 CONTINUE
130 N = 0
140 IF DRIVE IS IDLE GOTO 110
150 DELAY 50
160 N = N + 1
170 IF N < 20 GOTO 140
180 HALT DRIVE
190 BACK 15
```

PGM[38]

```
200 STEER LEFT 70 UNITS
210 IF RANDOM > 127 GOTO 240
220 GO BACKWARD 15 STEPS
230 GOTO 250
240 GO FORWARD 15 STEPS
250 STEER RIGHT 70 UNITS
260 GOTO 110
```

Remarks: The compiler will skip any statement that begins with REM. ANDROTEXT is designed to be read easily with a minimum of comments. Try this:

```
10 REM NO MATTER HOW MUCH
20 REM I PUT IN COMMENT STATEMENTS,
30 REM THE COMPILER WILL IGNORE THEM,
40 REM AND JUST RETURN ME TO THE
50 REM EXECUTIVE MODE
```

PGM[39]

Making Your Robot Talk

We've saved the most fun for last. ANDROTEXT makes speaking as easy as using the display. It takes a little longer in compilation because it must manipulate disk files. The time it takes is pretty much independent of program length, so you will generally want to save adding your SPEAK statements until your program is near completion. I find it is best to use DISPLAY statements until my programs are debugged, and then substitute SPEAK statements. The reason vocabulary information is read in and out is to leave maximum space for your program.

Put the HER01 ANDROTEXT Demonstration Diskette in your VIC-1541 Disk Drive. Type in this:

```
10 SPEAK *ROBOTS ARE VERY RELIABLE*
20 SPEAK *NOTHING CAN POSSIBLY*
30 SPEAK *GO WRONG*
40 GOTO 30
```

PGM[40]

Fun, huh? But, the most amazing part is yet to come.

The Incredible Adaptive Vocabulary: The phonemes for those words were drawn from a 200-word vocabulary on the disk. But, that vocabulary is constantly changing. If a word is not in the vocabulary, the compiler will first inspect it for common suffixes: s, es, ed, er, en, ly, ic, al, ing, est, ful, etc. If it finds one of these suffixes, it will look up the root word and add the sound for the suffix. If it can't find the root word, it will inspect it for common prefixes: un, de, re, pre, post, anti, etc. If this fails, it will try combinations of prefixes and suffixes. In principle, this makes it capable of recognizing more than 10,000 different written words. Finally, when it's exhausted its bag of tricks, the compiler will ask you for the correct root word and its phonemes. If the word had no recognizable suffix or prefix, the compiler will only ask for the phonemes. Try this:

10 SPEAK *MARY HAD A LITTLE LAMB*

PGM[41]

The compiler should have had no trouble. Although Hero didn't read it very poetically, he should have been intelligible. Now try this:

10 SPEAK *MARY HAD A LITTLE DOG*

PGM[42]

Midway through compilation, the following message should have appeared.

DOG
NOT IN VOCABULARY
PHONEMES?

At this point, you look on page 19 of your ET-18 ROBOT Voice Dictionary, and type in:

1E3D1C

The compiler should continue, and when you finish, Hero should do a respectable job of saying, "Mary had a little dog."

Now try it all over again.

This time the compiler doesn't need to ask for the phonemes for "dog." It remembers them. Does this mean the compiler now has a vocabulary of 201 words? No. The compiler threw out one word, and replaced it with "dog." How does it decide which word to throw out? This depends on three factors: (1) recency; (2) frequency; and (3) chance. These are the same factors that affect human memory. The more recently a word has been used, the less likely it will be forgotten. The more frequently it is used, the longer it will be retained. Among the lowest ranking words, the choice of which will be eliminated is made at random. When you start to develop a program, you should write a starting vocabulary on the disk with the program. At first you will want to use the one on the demonstration disk. You will be surprised how quickly the vocabulary adapts to that program's peculiar vernacular.

Let's try something with prefixes and suffixes. Naturally, this example is a bit contrived.

10 SPEAK *PRETRAINING IS DIFFICULT*

PGM[43]

The compiler should come back with this message.

```
PRETRAINING
PRETRAIN
TRAIN
NOT IN VOCABULARY
CORRECT ROOT?
```

You then type in:

```
TRAIN
```

The compiler will further query:

```
PHONEMES?
```

Then you turn to page 69 of your voice dictionary and answer:

2A2B0621290D

Now the compiler will attach the sounds for the prefix and suffix and Hero will be able to read this innane sentence.

Changing the Sound of a Word: If you're dissatisfied with the sound of a word and you want to change it, you simply add the character "%" as a suffix. The next time you compile, you will be asked for new phonemes. Try this:

10 SPEAK *MARY HAD A LITTLE% LAMB*

PGM[44]

Midway through compilation you will receive the following message.

LITTLE
OLD PHONEMES 180B092A2318
NEW PHONEMES ?

You are now free to make up your own pronunciation and have it retained, or simply hit return to retain the old pronunciation.

ANDROTEXT Editing

So far you've learned how to write some simple programs, how to list them, how to compile them, and how to make minor changes. As your programs grow more complex, you will need to use more advanced features of the editor.

Loading Programs : Load the ANDROTEXT Editor and Compiler as you were instructed on page 3. Put the HERO1 ANDROTEXT Demonstration Diskette into your disk drive A, and type in:

LOAD

The editor will ask,

FILENAME?

Answer,

PROGRAM 33

PROGRAM 33 (PGM[33]) has been recorded on the demonstration diskette, and can be found on page 26. When the disk drive stops and the cursor returns, type,

LIST

You should obtain a listing of PROGRAM 33 of your video monitor.

Enter the edit mode by typing in:

EDIT

A new cursor will appear in the form of a white block. You can move it about on the text of PROGRAM 33 with the cursor arrows, and make insertions and deletions in the usual manner. An edited line is not actually entered into the text until you hit the carriage return. Experiment a bit. The editor has a number of features we think you will like.

Return to the command mode by hitting the ESCAPE key.

Listing Programs: Type in this command:

LIST 230

Your monitor should show statement number 230

Now type in the following:

LIST 170-210

You should now have a listing of only statements 170 through 210.
Try this:

LIST 147-223

There are no statements 147 or 223, so the editor listed all statements between.

Select a statement on the screen, and blank it out except for the statement number. Relist that entire portion of the program and notice that the statement you blanked out is entirely removed. You could have accomplished the same thing by just typing the statement number.

There are two more things to try. First, type this:

LIST -150

This gives you all the statements from the beginning of the program up to and including statement 150. Now try,

LIST 150-

It lists all the statements from 150 to the end of the program.

Hard Copies of Programs: If you have a printer attached, you can obtain hard copies of your programs by using the TYPE command. If your Centronics parallel printer connector is still attached to the robot interface, connect it to your printer. The TYPE command does exactly the same as the LIST command, but does it on paper. Try this:

TYPE 147-223

Saving Programs: Type in this:

SAVE

The editor will query,

FILENAME?

Now if you typed in PROGRAM 33, the PROGRAM 33 on your demonstration diskette would be replaced by your edited version. If you gave it some other name it would create a new file under that name. But try simply hitting the return key. The screen should show this:

FILENAME DEFAULT

Your edited version of PROGRAM 33 is now stored in the default file, which is a handy temporary storage for developing programs. It is always good to save the program you have under development in the DEFAULT file from time to time so you can back and fill, or retrieve it in case the system bombs.

Renumbering Programs: With PROGRAM 33 in the editor, try this:

RENUMBER

The editor will ask:

ADDEND?

For this example, type in 1000. When the editor returns with

RENUMBERING COMPLETE

type in

LIST

The number 1000 has been added to all the statement numbers and to the GOTOs and the GOSUBs. What's it good for? Stay tuned.

Merging Programs: Now, save this renumbered program in the DEFAULT file, and load PROGRAM 33. Use LIST to check that PROGRAM 33 has been properly loaded, and then type:

MERGE

When the editor asks:

2ND FILENAME?

just hit the return key. You could have specified any file you wanted, as long as its first statement number was greater than the last statement number of the program in the editor. The DEFAULT file is a good place to keep your merging program. In a few minutes the monitor will display,

MERGING COMPLETE

Now, list the whole thing. You can either save this, or ignore it and go on to the next example.

Tidying Programs: The trouble with any programming language that numbers every statement is sooner or later you want to squeeze a statement between two consecutive statements, and you're stuck having to rearrange a large portion of the program. This is not just a pain in the neck; it often introduces errors that you don't discover until long after you've made the change. The TIDY command is designed to circumvent this problem. Load PROGRAM 33 again, and then type the following:

TIDY

The editor will then ask,

1ST STATEM'T?

This gives you the opportunity to accomplish the same effect as RENUMBER, while you increase the space between those jambed up statements. For now just put in 1000. The next question will be:

INTERVAL?

Most programmers like to have intervals of 10, but for this example put in 100. The editor will grind away and eventually come back with:

TIDYING COMPLETE

List the result. Every statement is separated by an even 100, and would be regardless of how the original statements were numbered.

In general you will want to separate your program into logical subunits and perform TIDY operations on them individually. Only at the most auspicious moment will you want to MERGE them into the final product.

Pattern Seeking and Replacement in Programs: The editor also allows you to seek out and replace patterns of indefinite length. With the infamous PROGRAM 33 in the editor, type in:

SEARCH

The editor will come back with:

E = EXIT
C = CONTINUE
R = REPLACE PATTERN
PATTERN?

Just to keep it simple, type in

KLATU

The editor should come back with the statement in which that pattern first occurs and should highlight the first occurrence. Now you have the choice of replacing that pattern, continuing to the next occurrence without any change, or exiting the search routine. Just for laughs, let's replace it. Hit the "R" key. The editor should ask:

PATTERN?

Try entering:

HAROLD

Now list the program. You will find it even works on this silly replacement.

Opcode Editing
For Afficianados and Masochists Only

Listing Opcode: Load PROGRAM 8. You remember this one. Now compile it. This time, instead of transferring it, type in LIST. You should get this:

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00D0	83	86	00	B7	01	6A	86	05	B7	01	6B	B6	01	6A	BB	01
00E0	6B	B7	01	6A	B6	01	6A	B7	00	C8	BD	01	17	BD	01	63

```
00F0  BD 01 63 86 C8 B7 01 6C B6 01 6A B1 01 6C 24 03
0100  7E 00 D6 BD F6 5B BD F7 E5 47 30 15 30 5B B7 BD
0110  01 63 BD 01 63 3F 3A 4F B7 00 C9 B7 00 CA B7 00
0120  CB B6 00 C8 80 64 25 0C B7 00 C8 B6 00 C9 4C B7
0130  00 C9 20 ED B6 00 C8 80 0A 25 0C B7 00 C8 B6 00
0140  CA 4C B7 00 CA 20 ED B6 00 C8 B7 00 C8 BD F6 5B
0150  B6 00 C9 BD F7 B5 B6 00 CA BD F7 B5 B6 00 C8 BD
0160  F7 B5 39 CE FF FF 09 26 FD 39
```

That's called opcode. It is exactly what is transferred to Hero, after we compile the ANDROTEXT. The numbers on the left are in the hexadecimal numbering system, and tell where that particular byte (2 digits) appears in Hero's memory. You don't have to learn anything about this unless you want to.

Notice that we used the ~~same~~ command we used to list the ANDROTEXT before. What's different? We are now in opcode mode and all commands apply to opcode. You know this because the boarder of the monitor screen is light green. The boarder is dark blue if you are in ANDROTEXT mode. Now type this;

TEXT

and then,

LIST

You have returned to ANDROTEXT mode. Let's go back again. Type:

OPCODE

Hard Copies of Opcode: Connect your printer and type:

TYPE

Your printer should disgorge the following.

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00D0	83	86	00	B7	01	6A	86	05	B7	01	6B	B6	01	6A	BB	01
00E0	6B	B7	01	6A	B6	01	6A	B7	00	C8	BD	01	17	BD	01	63
00F0	BD	01	63	86	C8	B7	01	6C	B6	01	6A	B1	01	6C	24	03
0100	7E	00	D6	BD	F6	5B	BD	F7	E5	47	30	15	30	5B	B7	BD
0110	01	63	BD	01	63	3F	3A	4F	B7	00	C9	B7	00	CA	B7	00
0120	CB	B6	00	C8	80	64	25	0C	B7	00	C8	B6	00	C9	4C	B7
0130	00	C9	20	ED	B6	00	C8	80	0A	25	0C	B7	00	C8	B6	00
0140	CA	4C	B7	00	CA	20	ED	B6	00	C8	B7	00	CB	BD	F6	5B
0150	B6	00	C9	BD	F7	B5	B6	00	CA	BD	F7	B5	B6	00	CB	BD
0160	F7	B5	39	CE	FF	FF	09	26	FD	39						

It's the same thing you saw before, only spread out a bit for easy reading. Now connect up your robot and transfer this opcode. When Hero says, "Ready," and the cursor returns, go over to Hero and punch in the following on his keyboard.

AE 00 D0

The display should show

000083

The first four digits are the location of the byte (0000) and the last two are the byte in memory (83). Now check this against the hard copy. To advance to the next byte, hit the "F" key on Hero. Notice it's the next byte on the hard copy (86). If you didn't have the interface, you could program Hero by punching in the opcode one byte at a time. People who don't have ANDROTEXT not only have to punch in one byte at a time, but they have to figure out what this stuff means! Lucky you!

Saving Opcode: You can save opcode just like you save ANDROTEXT. List the opcode for PROGRAM 8 just to make sure it's there. Then type in:

SAVE

When asked for a filename, type in OPCODE8 to remind yourself that you're saving opcode. When the cursor returns, type in

END

which will return you to BASIC, or turn your computer off completely.

Loading Opcode: Load and run the ANDROTEXT editor and compiler. When the dark blue boarder appears, type in:

OPCODE

The boarder will turn light green. Type in:

LOAD

When asked for a filename, type in OPCODE 8. When the cursor

returns, type in LIST. You will find you have recovered the opcode for PROGRAM 8.

Saving opcode lets you transfer programs to your robot without recompiling each time. Remember: save and load opcode only in opcode mode (green boarder) and save and load ANDROTEXT only in ANDROTEXT mode (blue boarder).

For Real Dedicated Afficionados or Pathological Masochists

If, for some obscure reason, you are not satisfied with the opcode the ANDROTEXT compiler generates, you can change it. Be sure you know what you're doing. Here are some other features.

Partial Listing: This works like the ANDROTEXT lister except you specify a range of line numbers. Try this:

LIST 00E0-00F7

The editor should come back with:

00E0 6B B7 01 6A B6 01 6A B7 00 C8 BD 01 17 BD 01 63

00F0 BD 01 63 86 C8 B7 01 6C B6 01 6A B1 01 6C 24 03

Notice that the list included all the line numbers enclosed. You get the same kind of thing if you type:

TYPE 00E0-00F7

Try fooling around with this a bit until you get the idea of how it works.

Pattern Seeking and Replacement in Opcode: Enter the following:

SEARCH

You should get this back:

E = EXIT
C = CONTINUE
R = REPLACE PATTERN
PATTERN?

Look familiar? And, you guessed it, you can now replace any pattern in the opcode. Try typing this:

00

You should get a block of continuous opcode about 50 characters long with the first occurrence of 00 highlighted. This happens to be the hexadecimal representation of the number to which N is set in the first line of PROGRAM 8, namely 0. Try replacing this with the following:

64

That's hexadecimal for 100. Now transfer this program to Hero and run it. This time Hero starts his count at 100.

If you really want to do a lot of opcode editing, remember that the blocks of opcode will be displayed without blanks and your replacement patterns should have no blanks. Also, you should be careful that you replace an even number of characters with an even number of characters. Opcode editing is tedious and difficult. Be sure you know what you're doing.

Special Features

Compiling Long Programs: If you compile a long program, more the 200 lines or so, you may run out of memory. The appropriate error message will be very obvious. Try reloading your program and type in:

LONG COMPILE

This will destroy the parts of your ANDROTEXT program, with which the compiler has finished, to make room for more opcode. When you return to ANDROTEXT mode, your program will be gone, so be sure to save it before starting the long compile. Also, be sure to clear memory by typing NEW frequently when dealing with long programs.

Compiling Humungous Programs (Linked Compiling): Your ANDROTEXT compiler is capable of compiling programs of indefinite length. You can use this capability by breaking a long program into sections. The last statement of each section must specify the filename of the next section to be compiled. For example:

```
10 REM THIS IS SECTION 1
20 SLOWLY MOVE FORWARD 10 STEPS
.. .....
.. .....
90 LINK SECTION2

100 REM THIS IS SECTION 2
... .....
... .....
500 LINK SECTION3

510 REM THIS IS SECTION 3
... .....
... .....
950 STOP
```

If these three sections were stored on disk with filenames SECTION1, SECTION2, and SECTION3, you could compile them as one program by simply loading SECTION1 and compiling as normal. The compiler would then create three files named LINK1, LINK2, and LINK3 containing the opcode. Transfer to HERO is done in the usual way, but you will notice that the compiler is reading the disk files. Your last section should be only a few statements long: it is best to make it simply a single remark. Each time you compile a linked program, you should type in NEW before loading the first file.

Location Maps: You can obtain a location map of your variables by simply typing in MAP when in opcode (green) mode. The map is only available after compiling.

Changing Disk Drive: The default disk drive is A. If you want to change this, simply type in

CHANGE

while in ANDROTEXT mode. The compiler will then query:

DISK DRIVE DEVICE?

If you want to change to drive B, type in B (return) or simply (return) for no change.

Generating a New Vocabulary File: If for some reason you lose all your old vocabulary files, you can always use the editor to generate a new one. In ANDROTEXT mode, just type in:

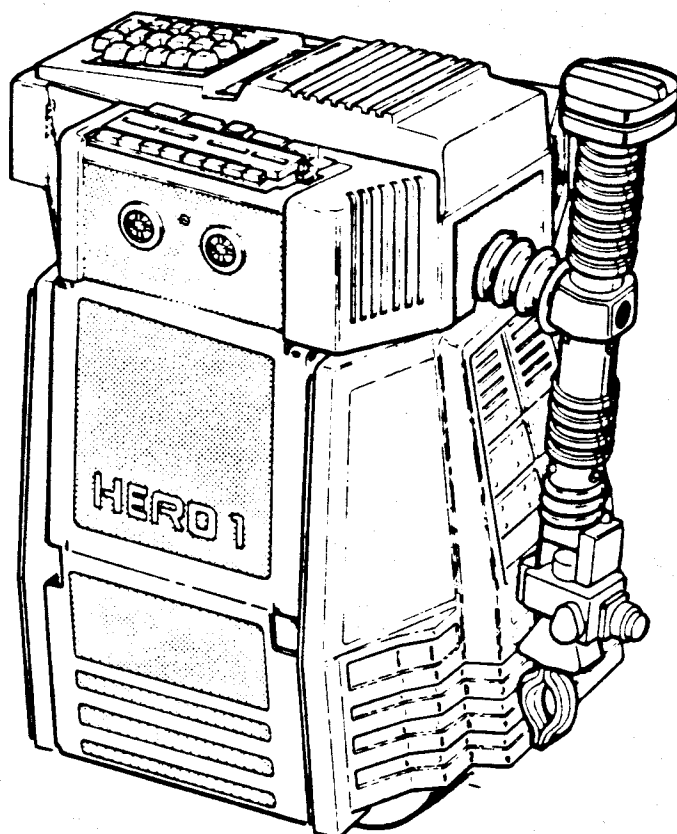
VOCABULARY

When the cursor returns, put your program development diskette into the disk drive, and save (as if you were saving an ANDROTEXT program) under the filename VOCABULARY. The monitor display should look like this.

SAVE
FILENAME?VOCABULARY
FILENAME VOCABULARY

VOCABULARY is a file in the same format as any ANDROTEXT program. You should not try to store a program with the filename VOCABULARY. If you do, you will clobber the vocabulary.

The new vocabulary will contain only one word: nothing. It will behave like a ordinary vocabulary file, and will ask you for the phonemes of new words as it encounters them.



Appendices

APPENDIX 1Display Code For Characters

These are the characters recognised by ANDROTEXT and the opcode and display pattern for Hero-1. All other ASCII characters result in blanks.

<u>Character</u>	<u>Opcode</u>	<u>Display</u>	<u>Character</u>	<u>Opcode</u>	<u>Display</u>
space	00		G	7B	9
!	A0	!	H	37	H
(	4E	[	I	30	I
)	7B	]	J	3C	J
,	80	.	K	0F	k
-	01	-	L	0E	L
.	80	.	M	76	n
/	30	/	N	15	n
0	7E	0	O	7E	O
1	30	1	P	67	P
2	6D	2	Q	73	q
3	79	3	R	05	r
4	33	4	S	5B	S
5	5B	5	T	70	7
6	5F	6	U	1C	u
7	70	7	V	3E	U
8	7F	8	W	36	W
9	7B	9	X	49	=
A	77	A	Y	33	Y
B	1F	b	Z	6D	2
C	4E	[	[	4E	[
D	3D	d	£	0E	L
E	4F	E	]	7B	]
F	47	F			

ANDROTEXT 1.0 QUICK REFERENCE

VARIABLES

Any length, but contain no blanks, numbers, or special characters except hyphen [-]

Integer 0 to 255
Logical True or False

TABLES

Same as variables except end in \$
Sum of table lengths 256

TABLE AS n Creates table AS with n entries
DATA n n o Assigns data to table
ENTRY Reserved variable gives table entry

ARITHMETIC OPERATORS

= Assign value to variable
+ Addition
- Subtraction
* Multiplication
/ Division

LOGICAL OPERATORS

AND Logical "And"
OR Logical Inclusive "Or"
XOR Logical Exclusive "Or"
NOT Logical "Not"
A RIGHT n Shifts A right n places
A LEFT n Shifts A left n places

Bits emptied by shifts fill with 0s

RELATIONAL OPERATORS

= Equal
<> Not Equal To
< Less Than
> Greater Than
>= Less Than or Equal to
<= Greater Than or Equal to
IS State of Being

SYSTEM COMMANDS

All system commands may be abbreviated by first two letters

COMPILE Compiles ANDROTEXT program in memory
LOAD FILENAME?A Loads program A from disk
OPCODE Changes system to opcode mode
SAVE FILENAME?A Saves program A to disk
TEXT Changes system to ANDROTEXT mode
TRANSFER Transfers opcode to robot

ANDROTEXT EDITING COMMANDS

LIST Lists entire program
LIST n Lists line n
LIST n-m Line n to line m
LIST -m Beginning to line m
LIST n- Line n to end

MERGE Attaches program A on disk to program in memory
2ND FILENAME?A

REM A Comment message A listed but ignored in compilation

RENUMBER Adds n to all statement numbers
ADDEND?n

SEARCH Searches program for pattern A, displays each occurrence, and permits replacement with pattern B
PATTERN?A
REPLACEM'T?B

TIDY Sets first statement number to n, and statement number intervals to m
1ST STATEM'T?n
INTERVAL?n

TYPE Same as LIST command
TYPE n but outputs to printer
TYPE n-m
etc

OPCODE EDITING COMMANDS

LIST Lists opcode
LIST n n 5 m are locations in robot's memory
LIST n-m Each line 8 bytes
etc

SEARCH Searches opcode for pattern A, displays each occurrence, and permits replacement with pattern B
PATTERN?A
REPLACEM'T?B

TYPE Types opcode
TYPE n Each line 16 bytes
TYPE n-m
etc

ACTIVE SENSORS

SENSE A Activates Sense A
IGNORE A Deactivates Sense A

sense reserved variables
DATE YEAR, MONTH, DAY
INTERRUPT INTERRUPT
KEYBOARD KEYBOARD
LIGHT LIGHT
MOTION MOTION
RANGE RANGE, HITS
SOUND SOUND
TIME HOUR, MINUTE, SECOND
TRIGGER TRIGGER

PASSIVE SENSORS

reserved variables	range
ARM-EXTEND	0-152
ARM-PIVOT	0-134
GRIPPER-OPEN	0-117
HEAD-ANGLE	0-181
STEERING-ANGLE	0-152
WRIST-PIVOT	0-181
WRIST-ROTATE	0-147

ACTIVE EFFECTORS

Recognizes most unambiguous commands
These are always recognized

ARM	EXTEND/RETRACT	
	PIVOT	UP/DOWN
DRIVE	FORWARD/BACKWARD	
GRIPPER	OPEN/CLOSE	
HEAD	LEFT/RIGHT	
STEER	LEFT/RIGHT	
WRIST	ROTATE	LEFT/RIGHT
	PIVOT	UP/DOWN

ROTATE WRIST LEFT
ROTATE WRIST LEFT 3 UNITS
ROTATE WRIST TO 16

PASSIVE EFFECTORS

DISPLAY *HELLO*	LEDs show "HELLO"
DISPLAY n	LEDs show number n
SPEAK *HELLO*	Says "Hello"
SPEAK n	Says number n

MISCELLANEOUS COMMANDS

DELAY n	Delays n milliseconds
HALT A	Stops effector A
INITIALIZE	Initializes motors
PAUSE n	Delays n seconds
ROLL	Starts random numbers
SLEEP n	Sleeps n minutes
STOP	Returns to executive

PROGRAM FLOW

GOTO n	Branches to line n
GOSUB n	Branches to subroutine starting at n
RETURN	Marks end of subroutine. Returns to statement following most recent GOSUB

IF A = 3 GOTO n	If assertion is true, take branch
IF A = 3 GOSUB n	If false, execute next line
IF ARM IS	
IDLE GOTO n	

GENERAL

Single blank is only separator

Variables/Tables must be assigned in previous statement before used

Arithmetic and logical operations limited to single operation left of =

Warranty and Copyright Information

Warranty Disclaimer: The computer programs supplied on the magnetic diskette and these instructions are provided to you, their user, with no warranty of any kind. Although due care has been taken to ensure the correct operation of these programs, no representation is made about their fitness for any particular use or about the accuracy of their results. Robotronix Inc., their distributors, retailers, and agents thus assume no responsibility and accept no consequential, incidental, or other liability arising from the use of these programs. Some states do not allow the exclusion or limitation of implied warranties or liability for incidental or consequential damages, so the above limitations may not apply to you.

All ANDROTEXT Editor and Compiler Diskettes are thoroughly inspected and tested before final packaging and are covered by a limited 90-day warranty on the diskette. It is still possible, however, to experience loading problems with a known good diskette. If you are having problems, please try the procedures in the section titled In Case of Trouble.

If you are still unable to load and run the ANDROTEXT Editor and Compiler Diskette, you can contact your dealer for assistance. If you experience further difficulties during the 90-day warranty, you should send the Diskette and proof-of-purchase directly to:

Robotronix Inc.
Box 1125
Los Alamos, NM 87544

We will replace your Diskette at the appropriate exchange fee and return it to you via First Class Mail. If, after receiving this replacement Diskette, you continue to experience loading difficulties, your disk drive may require alignment or adjustment. Contact your dealer for additional information.

Specifications and prices are subject to change at any time and without notice.

Copyright Notice: The computer programs supplied on the magnetic diskettes and User's Guide are covered by international copyrights. Reproduction by any method whatsoever of these original copyrighted programs and/or instructions is expressly prohibited without written

permission from Robotronix Inc.

INDEX

Active Effectors	17	Getting Started	3
Active Sensors	12	HALT.....	21
Adaptive Vocabulary	28	Hard Copies, ANDROTEXT ..	33
ANDROTEXT	3	Hard Copies, Opcode	38
ANDROTEXT Editing	31	HER01	3
Hard Copies	33	Ignoring Senses	17
Listing	31	INITIALIZE	26
Loading	31	INTERRUPT	16
Merging	34	KEYBOARD	17
Pattern Seeking		LIGHT	15
and Replacement	36	Listing ANDROTEXT	32
Renumbering	34	Listing Opcode	37
Saving	33	Loading ANDROTEXT	31
Tidying	35	Loading Opcode	40
ANDROTEXT Statements.....	3	Logical Operations	9
Arithmetic Operations	8	Making Your Robot Talk ..	28
Assignments	8	Merging ANDROTEXT	35
Bit Picking	11	Miscellaneous Commands	
Branching	7	that Make Life Easy	25
Changing the		MOTION	14
Disk Drive	43	Operations, Arithmetic ...	8
Sound of a Word	30	Operations, Logical	9
Characters, Display Code	44	Opcode Editing	37
Conditional Branches	8	Hard Copies	38
Conditional		Listing	37
Branches on Effectors ...	22	Loading	40
CONTINUE	20	Partial Listing	41
Data	11	Pattern Seeking	
DATE	16	and Replacement	41
DELAY	25	Saving	39
Device Number	42	Partial Listing Opcode...	40
Display Code Characters .	46	Passive Effectors	22
Editing, ANDROTEXT.....	31	Passive Sensors	21
Editing Opcode	37	Pattern Seeking and	
Effectors, Active	18	Replacement, ANDROTEXT ..	36
Effectors,		Pattern Seeking	
Conditional Branches on .	23	and Replacement, Opcode..	41
Effectors Passive	22	PAUSE	25
Generating		Quick Reference	46
a New Vocabulary File ...	42	Random Numbers	27
		RANGE	13
		Remarks	28
		Renumbering ANDROTEXT ...	34

INDEX CONTINUED

Reserved Variables	13	Tables	10
.....	21	Tables in	
Saving ANDROTEXT	33	Effector Commands	19
Saving Opcode	40	Talking	28
Sensors, Active	12	Tidying ANDROTEXT	35
Sensors Passive	21	TIME	16
Shift Commands	11	TRIGGER	17
SLEEP	26	Trouble	43
SOUND	14	Variables	7
Sound of a Word	30	Variables in	
Special Features	42	Effector Commands	19
STOP	27	Vocabulary, Adaptive	28
Subroutines	23	Vocabulary File,	
		Generating	42